

Using Built-in Styles

Matplotlib has a number of predefined styles available. These styles contain a variety of default settings for background colors, gridlines, line widths, fonts, font sizes, and more. They can make your visualizations appealing without requiring much customization. To see the full list of available styles, run the following lines in a terminal session:

```
>>> import matplotlib.pyplot as plt
>>> plt.style.available
['Solarize_Light2', '_classic_test_patch', '_mpl-gallery',
--snip--
```

To use any of these styles, add one line of code before calling `subplots()`:

```
mpl_squares.py import matplotlib.pyplot as plt

input_values = [1, 2, 3, 4, 5]
squares = [1, 4, 9, 16, 25]

plt.style.use('seaborn')
fig, ax = plt.subplots()
--snip--
```

This code generates the plot shown in Figure 15-4. A wide variety of styles is available; play around with these styles to find some that you like.

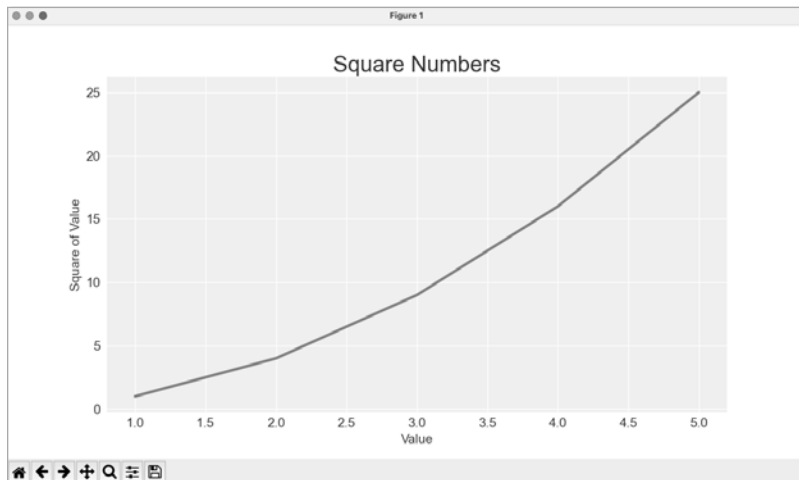


Figure 15-4: The built-in seaborn style

Plotting and Styling Individual Points with `scatter()`

Sometimes, it's useful to plot and style individual points based on certain characteristics. For example, you might plot small values in one color and larger values in a different color. You could also plot a large dataset with one set of styling options and then emphasize individual points by replotting them with different options.